Pokédex Search

A Coveo FDE technical challenge

Franck Benichou · Forward Deployed Engineer candidate

🔍 Live app · pokemon-search-one-chi.vercel.app

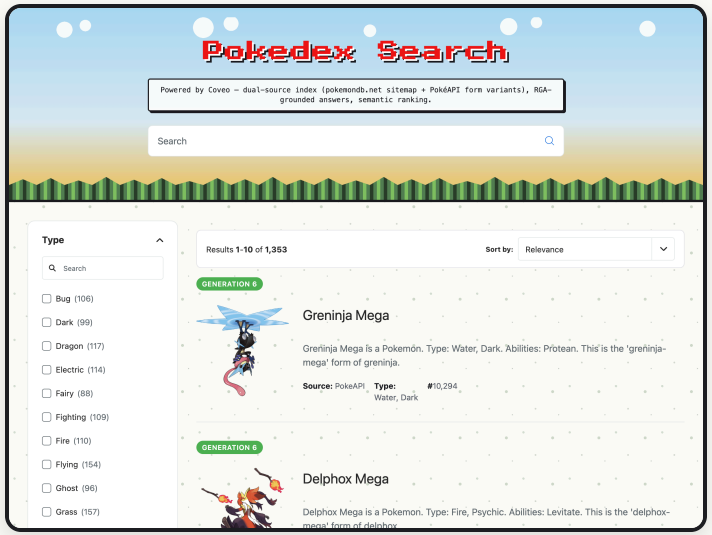
💻 GitHub · github.com/benichou/coveo-pokemon-challenge

🔬 RGA performance monitoring · pokemon-rga-dashboard.vercel.app

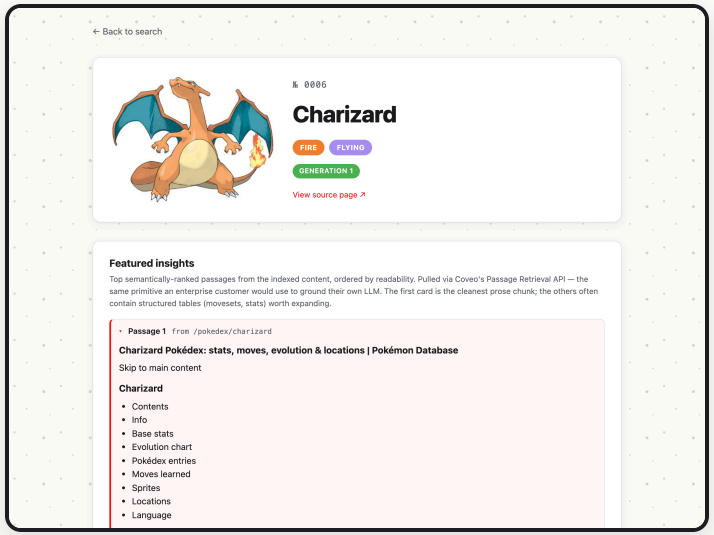
📊 Query observability · grafana public dashboard

What I built

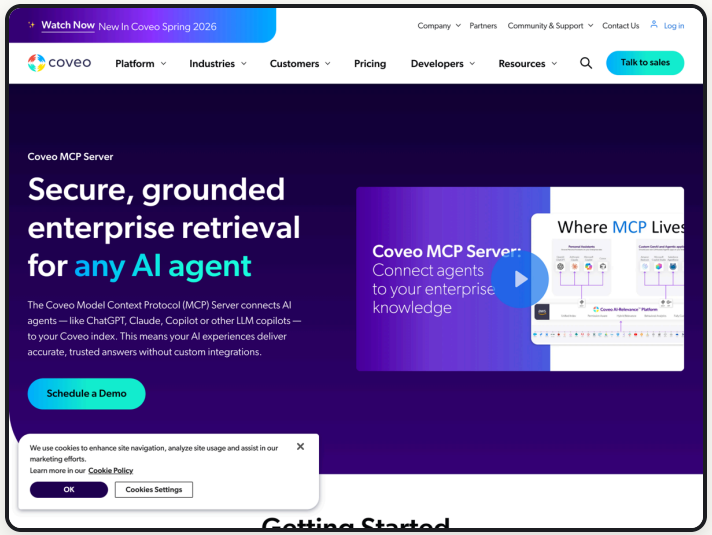
A Pokémon-themed search experience on Coveo Cloud — **three user-facing surfaces** powered by **one Coveo brain**.



Atomic main page



Pokémon Detail Page



Coveo MCP Server

github.com/benichou/coveo-pokemon-challenge

Architecture · one Coveo org, three UI surfaces, two loops

📚 Data sources

pokemondb.net
sitemap · 12,915 URLs · public site

PokéAPI
per-form variants · Mega / Hisuian / Galarian

→

🛠️ Ingestion · 95% as-code

Source A · Coveo Sitemap source
1,028 docs · versioned scraping config

Source B · Python Push pipeline
+325 form variants · enrichment via PokéAPI

→

🧠 Coveo Cloud Org · benichou

Unified index
1,353 docs · 5 indexed fields · least-privilege API keys

4 ML models on default pipeline
RGA · Semantic Encoder · Query Suggest · Passage Retrieval

↓

🎯 Three UI surfaces · one retrieval brain

Atomic main page

`/`

list view · RGA panel · PR panel · Query Suggest type-ahead · GBC-themed UI

Pokémon Detail Page

`/pokemon.html?name=X`

Headless + React · three parallel Coveo queries (hero · passages · related)

Coveo MCP Server

`coveo-pokemon`

4 MCP tools (search · fetch · get_passages · answer) addressable from any MCP client

📊 Query observability loop

Atomic → Vercel proxy → Grafana Cloud Loki → Public dashboard

↑ log every search · fire-and-forget · token stays server-side

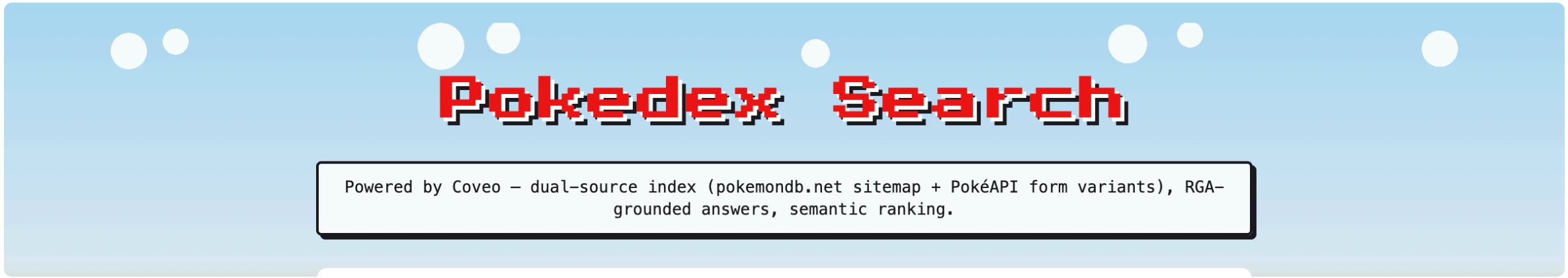
🔬 Continuous AI quality (closed loop)

Daily eval → analyzer (5-run window) → 5 guardrails → apply via Coveo ML Models API ↺

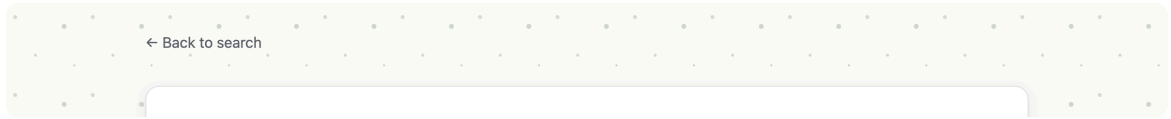
↑ closed loop · auto-rollback on next-day drop > 5pts · audit trail per run

github.com/benichou/coveo-pokemon-challenge

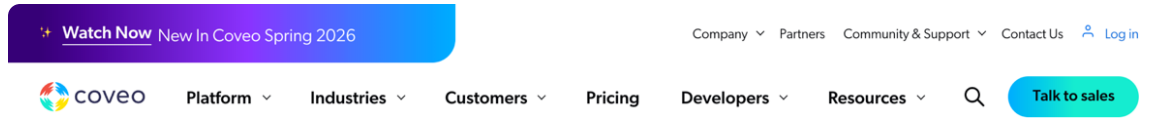
Live demo · switching to the browser



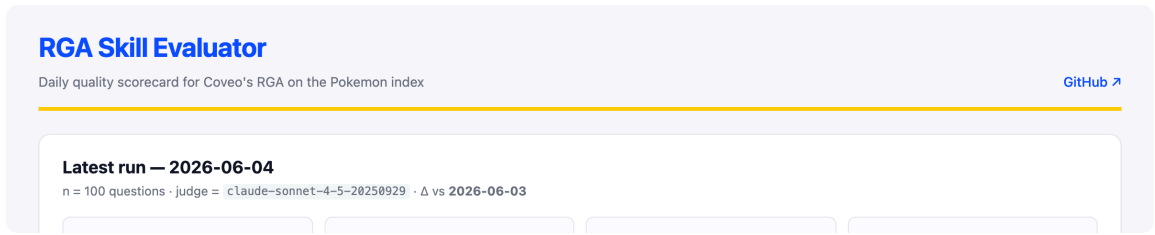
🔍 Atomic main page · `pokemon-search-one-chi.vercel.app`



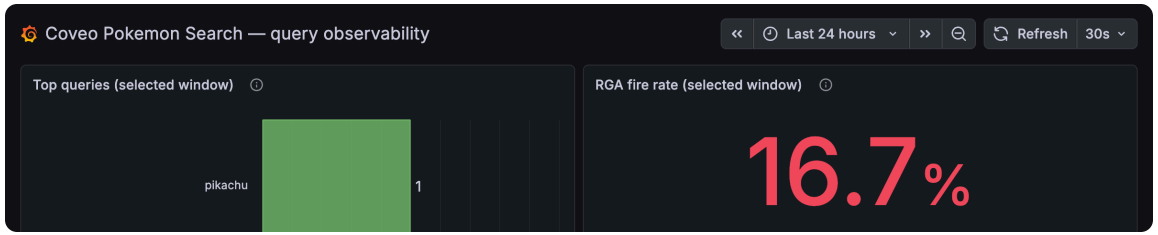
Pokémon Detail Page (Headless + React)



Coveo MCP Server (coveo-pokemon)



RGA performance monitoring



Query observability

Follow along — full source at **`github.com/benichou/coveo-pokemon-challenge`**

github.com/benichou/coveo-pokemon-challenge

Why dual-source ingestion

The **1,353 docs mixing in the result list** *during the demo — that's two ingestion sources unified into one Coveo index.*

	Source A · Coveo Sitemap source	Source B · Python Push pipeline
Effort	Zero code · just a versioned scraping config	Python pipeline (`push-pokemon/`)
Throttling	Coveo handles automatically	I handle (PokéAPI 100 req/min cap)
Freshness	Refresh on a schedule	On-demand re-push
Form variants	One doc per canonical slug	One doc **per form** (Mega · Hisuian · Galarian)
Use for	Canonical Pokémon pages	Per-form coverage + PokéAPI enrichment

Not *"one source is right"* — different sources serve different facets of the same data.

github.com/benichou/coveo-pokemon-challenge

Coveo config — code vs Console

Everything in the demo's search behavior is reproducible from code. Coveo gates a small set of one-time setup steps to the Console.

✅ Code-as-source-of-truth

```
config/                       · Coveo resource declarations
├── fields.json               · 5 indexed fields
├── source/                   · Sitemap A: definition · scraping · URL filter
├── ml/                       · QS seed CSV (152 weighted queries)
└── mcp/pokemon-mcp.yaml      · MCP Server config (manually mirrored)

scripts/                      · bash · Python · idempotent · REST apply
├── bootstrap.sh              · one-command full provisioning
├── validate/                 · API keys + org features preflight
├── setup/                    · fields · source · scraping · mappings
├── source/                   · widen filter · rebuild · poll
├── ml/                       · associate 4 models · seed Query Suggest
├── mcp/discover_api.sh       · probes for admin REST API (404 confirmed)
└── audit/                    · PokéAPI cross-ref · index leak detection

push-pokemon/                 · Python pipeline · Source B implementation
                                scrape pokemondb · enrich via PokéAPI ·
                                transform · POST to /push/v1/.../documents

rga-eval/                     · 100-Q golden + Sonnet 4.6 judge · drives loop
rga-closed-loop/              · daily prompt tuning
├── prompts/pokemon-rga.yaml  · RGA Custom Prompt (PUT to ML Models API)
├── prompts/history/          · every prior version, dated YAMLs
├── src/apply.py              · dry-run default · --apply to write
├── src/closed_loop_run.py    · cron orchestrator
└── src/guardrails.py         · confidence · lift · rate-limit · sanity

.github/workflows/            · autonomous cron-driven Coveo updates
├── rga-eval-daily.yml        · 06:00 UTC · eval + bot-commit
└── closed-loop-daily.yml     · 06:30 UTC · analyzer + apply via API
```

github.com/benichou/coveo-pokemon-challenge

⚠️ Console-only (Coveo product design)

- **Org creation** — Coveo vendor onboarding flow; can't bootstrap via API
- **6 API keys** — Coveo shows secrets once at creation, by design (one-time per key)
- **ML model creation** — RGA · SE · QS · PR. Console-only per Coveo docs. Only the *post-creation behavior* (prompts, seed queries, pipeline associations, builds) is API-scriptable.
- **MCP Server creation + admin** — no public admin REST API yet (verified via `scripts/mcp/discover_api.sh` — 8 candidate endpoints all 404). `config/mcp/pokemon-mcp.yaml` is the source-of-truth; the Console is currently a manual mirror.

Four ML models · one pipeline

All four models attached to one query pipeline — the challenge brief asks for RGA and Query Suggest; I added the other two because the index already supports them.

▸ RGA

Relevance Generative Answering

LLM-grounded answer + citations above the result list.

Custom Prompt versioned in `rga-closed-loop/prompts/pokemon-rga.yaml` · **v1.1.0 live** · 8 rules

↳ *The prompt itself is autonomously tunable — see closed-loop slide*

▸ SE

Semantic Encoder

Embedding-based relevance — query/doc similarity beyond keyword overlap.

Console-created · associated via `scripts/ml/associate_models.sh` · no prompt config needed

↳ *Recovers "what type is charizard" even when the doc says "Fire/Flying"*

▸ QS

Query Suggest (type-ahead)

In-search-box suggestions as the user types.

152 weighted seed queries in `config/ml/default-queries.json` · CSV PUT to Coveo's Default Queries endpoint

↳ *Solved cold-start without UA event synthesis · see* `docs/ml-models.md`

▸ PR

Passage Retrieval (bonus)

Returns ranked text passages — chunked grounding primitive.

Console-created · associated to default pipeline · powers two UI surfaces (Atomic + Detail Page)

↳ *Same primitive an enterprise would use for CRGA — bridge to Topic 2*

github.com/benichou/coveo-pokemon-challenge

Three UI surfaces + cross-cutting ops skills

Same Coveo brain · different consumption modes. **The third surface (MCP) is for AI agents — that's the 2026 story.** `/pokemon-mcp` *is its 1:1 driver (in the MCP card). The other two skills sit across all three surfaces.*

BROWSER USERS

Atomic main page

List view · facets · RGA panel · Passage Retrieval · Query Suggest type-ahead · GBC-themed UI.

Vite + `@coveo/atomic` · 5 rotating biome themes · Press Start 2P pixel font

Drive it: open `pokemon-search-one-chi.vercel.app`

BROWSER USERS

Pokémon Detail Page

Hero · Featured insights (PR) · Related grid — three parallel Coveo round-trips on one page.

Vite + `@coveo/headless` + React · two Headless engines · multi-entry build

Drive it: click any Atomic result, or `/pokemon.html?name=X`

AI AGENTS

Coveo MCP Server

4 tools — `search` · `fetch` · `get_passages` · `answer` — addressable from any MCP client.

Coveo Hosted MCP · per-org endpoint · server instructions = LLM system prompt

Drive it: `/pokemon-mcp demo` in Claude Code

↑ TWO CLAUDE CODE SKILLS OPERATE ACROSS **ALL THREE SURFACES ABOVE** — THEY'RE FDE OPS, NOT NEW SURFACES

`/rga-eval`
Measure RGA answer quality continuously — see next slide

`/rga-closed-loop`
Tune the RGA prompt autonomously — see slide after that

github.com/benichou/coveo-pokemon-challenge

rga-eval · measure RGA quality every day

Bonus functionality. *Most enterprise AI deployments ship, regress silently, retune in Slack threads. I built the alternative — a continuous quality scorecard that runs on its own.*

▸ CRON
06:00 UTC daily

GitHub Actions · `rga-eval-daily.yml`

→

▸ GOLDEN
100 Q · 3 layers

50 single-fact · 35 multi-doc synthesis · 15 edge / refusal

→

▸ RUN
POST to Coveo RGA

Streaming `/answer/v1/.../generate` · 100 round-trips · ~10 min

→

▸ JUDGE
Sonnet 4.6 + deterministic

Accuracy + Precision (LLM judge) · Hard Recall (substring)

→

▸ ARCHIVE
JSON in repo

`eval-runs/YYYY-MM-DD-full.json` · commit history *is* the time-series DB · diff-reviewable

→

▸ DASHBOARD
Public · Vercel-hosted

`pokemon-rga-dashboard.vercel.app` · time-series · per-Q drill-down · prompt-change markers

↓ *This JSON archive is the* **input that drives the next slide** *— the closed loop reads the 5-day eval window every morning at 06:30 UTC*

▸ DRIVEN FROM CLAUDE CODE VIA

```
/rga-eval
Run a full 100-Q evaluation · show the latest results · drill into per-question outcomes
```

github.com/benichou/coveo-pokemon-challenge

rga-closed-loop · the system tunes itself

Measurement without action is just a dashboard nobody reads. The closed loop **acts** *on the eval signal — and only when 5 guardrails pass.* **More bonus functionality.**

↑ **Reads what the previous slide produces** — *yesterday's eval-run JSON · without that, this loop has nothing to act on*

▸ READ

5-day eval window

Per-category persistence + drift · samples failing answers from latest run only

→

▸ ANALYZE

Sonnet 4.6 · tool-use forced

Worst categories → structured `PromptProposal` (rationale + predicted lift + sample after-answer)

→

▸ PROPOSE

New prompt YAML

`prompts/pokemon-rga.yaml` v_{N+1} · archive prior to `prompts/history/`

→

▸ APPLY

PUT to Coveo ML Models API

`extraConfig.additionalAnswerInstructions` · audit log + git commit by the bot

↺ Tomorrow's 06:00 UTC eval measures the lift · the loop repeats

▸ FIVE GUARDRAILS BLOCK ANY UNSAFE APPLY — CONFIDENCE · LIFT · RATE-LIMIT · SANITY · AUTO-ROLLBACK

- Confidence ≥ 0.80
- Predicted lift ≥ +5pts
- Rate-limit ≥ 3 days between applies
- Sanity (no empty / 2× length)
- Auto-rollback if next-day drops > 5pts

▸ DRIVEN FROM CLAUDE CODE VIA

`/rga-closed-loop`

Run analyzer locally · dry-run apply · inspect audit log · rollback to previous version

github.com/benichou/coveo-pokemon-challenge

What I learned

Three lessons that actually changed how I'd approach the next Coveo engagement.

▸ LESSON 1

Inspect the data *before* choosing the ingestion strategy.

Pokémon look uniform at first — name · type · dex number. They're not. pokemondb publishes **one canonical page per slug**; PokéAPI exposes **per-form endpoints** for Mega-X, Mega-Y, Hisuian, Galarian variants. The dual-source approach — Sitemap for canonical pages, Push for form variants — fell out of an hour looking at actual HTML, not from a design pattern.

↳ *Ingestion architecture is a data question, not a tooling question · source choice = data inspection × downstream use case.*

▸ LESSON 2

Closed loops compound. Static prompts decay.

The daily eval + auto-tuning machinery felt like over-engineering before it ran. Once the cron started firing every morning, the cognitive load of "is the prompt still good?" went to zero — the system answers that for me.

↳ *Build the measurement loop FIRST · you only know a prompt is "final" by measuring it over time.*

▸ LESSON 3

Vector + LLM doesn't model *relationships* — that's the 2026 frontier.

Some queries are relational — *"which Fire-types evolve from Water-types?"*, *"which abilities counter Fire moves?"*. SE embeds entities; RGA composes language — neither models the **edges between entities**. Coveo's stack today (Catalog = commerce-only · Knowledge Hub = content mgmt · no GraphRAG) leaves multi-hop reasoning to a future layer.

↳ *Multi-hop questions will need a knowledge-graph layer next · a natural 2026 extension of the RGA story.*

github.com/benichou/coveo-pokemon-challenge

Production hardening · what I'd ship next

The build runs and behaves. **It's not production-grade yet.** *Four honest gaps below — each one is well-known engineering work, not invention.*

▸ HOSTING · SCALE · CONCURRENCY

Vercel → private cloud network

Today: Vercel auto-scales when requests arrive · Coveo handles their side · single region · first request after idle is slow ("cold start").

Gap: Never tested with simulated heavy traffic — simultaneous-user ceiling unknown · no fallback if our region goes down · no private network isolation.

↳ *Next: simulated-traffic tests to find the limit · AWS managed functions + load balancer + CDN in a private network · multiple regions at once · explicit uptime + speed targets.*

▸ AUTH · DATA ACCESS · SECURITY

API keys → short-lived per-user tokens

Today: 6 narrowly-scoped API keys in `.env` / Vercel / GitHub · public site, no login · MCP uses a shared key.

Gap: Frontend uses **API key directly** instead of a per-user token · no login (Okta/Azure AD) · no **per-user document filtering** from source systems (Salesforce/SharePoint/Confluence) · keys in env files, not a vault · no rotation.

↳ *Next: backend mints a fresh token per user · Okta/Azure AD single sign-on ·* ***Coveo Security Identities*** *— pull each user's source permissions at index time, enforce at search · vault for keys · quarterly rotation.*

▸ MONITORING BEYOND AI QUALITY

2 dashboards → full app monitoring + targets

Today: AI-answer-quality dashboard + Grafana query logs · rest is Vercel defaults.

Gap: No app-performance tracing (where time goes in each request) · no error tracking that pings me when something breaks · no scheduled bots that fake-visit to detect downtime · **no documented uptime targets** · no automated paging.

↳ *Next: app-performance monitoring (Datadog or Tempo+Grafana) · error tracking (Sentry) · scheduled uptime checks · written uptime + quality targets · automated paging when targets break.*

▸ AUTONOMOUS LOOP · OPERATIONAL TRUST

Can we trust the loop to run unattended?

Today: 5 safety checks · automatic rollback if next-day quality drops · new prompts go live to *100% of users immediately* when checks pass.

Gap: Safety checks never triggered by a real bad prompt — we don't know they actually fire · daily eval scores 100 questions *we wrote*, doesn't measure if real users *click* the answers · no alert per apply (silent commit).

↳ *Next: deliberately push a bad prompt to prove rollback works · roll out new prompts* ***10% → 50% → 100%*** *via Coveo's A/B framework (real user clicks decide the winner) · Slack/PagerDuty alert per apply.*

▸ MORE COVEO PLATFORM DEPTH I'D ACTIVATE AT SCALE

ART · clicks-trained relevance | **DNE** · ML-tuned facets | **IPE** · index-time Python | **UA Data Service** · native analytics export | **Catalog** · entity hierarchies

github.com/benichou/coveo-pokemon-challenge

Thank you .

Q&A

Three UI surfaces · two loops · one Coveo brain

Built as code · measured continuously · ready to tune itself overnight

🔍 Live app · pokemon-search-one-chi.vercel.app

💻 GitHub · github.com/benichou/coveo-pokemon-challenge

🔬 RGA performance monitoring · pokemon-rga-dashboard.vercel.app

📊 Query observability · grafana public dashboard

Appendix · References

Every Coveo claim in the deck links to its source · external references are listed for landscape context.

▸ COVEO DOCUMENTATION

SECURITY & DATA ACCESS (SLIDE 12)

- Management of security identities & item permissions
- Security identity cache and provider
- Content security options

A/B TESTING & ML ROLLOUT (SLIDE 12)

- Manage A/B tests
- Manage model associations with query pipelines

ML MODELS (SLIDES 7, 12)

- About Automatic Relevance Tuning (ART)
- Associate a DNE model with a pipeline
- Associate a QS model with a pipeline
- Inspect Coveo ML models

KNOWLEDGE GRAPH / GRAPHRAG STATUS (SLIDE 11)

- Coveo Knowledge Hub
- Catalog entity (commerce-only)
- Query extension for related entities

▸ EXTERNAL REFERENCES

GRAPHRAG 2026 LANDSCAPE (SLIDE 11)

- GraphRAG in 2026 — Practical Buyer's Guide
- Microsoft GraphRAG (research pipeline)
- WRITER Knowledge Graph (writer.com/product/graph-based-rag)
- Neo4j Agentic GraphRAG

DATA SOURCES (SLIDE 5)

- pokemondb.net sitemap (12,915 URLs)
- PokéAPI (per-form endpoints)

INFRASTRUCTURE & RUNTIME

- Vercel · hosting + serverless proxy
- Grafana Cloud · free-tier Loki for query logs
- GitHub Actions · daily eval + closed-loop crons
- Anthropic Claude API · Sonnet 4.6 (judge + analyzer)

TOOLING

- Marp (deck) · Mermaid (diagrams) · uv (Python) · Vite (frontend)

github.com/benichou/coveo-pokemon-challenge

Appendix · Repo map

Key files to open during/after Q&A — each anchors a specific claim from the deck.

rga-eval · continuous AI quality (slide 9)

- `rga-eval/golden/questions.json` · 100-Q dataset (50 / 35 / 15 layers)
- `rga-eval/src/main.py` · daily eval runner (~10 min)
- `rga-eval/src/llm_judge.py` · Sonnet 4.6 — Accuracy + Precision
- `rga-eval/src/metrics.py` · deterministic Hard Recall
- `eval-runs/*.json` · commit history = time-series DB

rga-closed-loop · autonomous tuning (slide 10)

- `rga-closed-loop/prompts/pokemon-rga.yaml` · live RGA prompt (v1.1.0)
- `rga-closed-loop/prompts/history/*.yaml` · every prior version archived
- `rga-closed-loop/src/analyzer.py` · Sonnet 4.6 PromptProposal
- `rga-closed-loop/src/guardrails.py` · 5 safety checks
- `rga-closed-loop/src/closed_loop_run.py` · cron orchestrator

atomic-search · three UI surfaces (slide 8)

- `atomic-search/src/main.js` · Atomic main page entry
- `atomic-search/src/pokemon-detail/App.tsx` · Headless + React detail
- `atomic-search/src/observability.js` · per-query Grafana logger
- `atomic-search/api/log-query.js` · Vercel serverless proxy

MCP · agent-facing surface (slide 8)

- `config/mcp/pokemon-mcp.yaml` · MCP server source-of-truth
- `.claude/mcp.json` · Claude Code HTTP MCP wiring
- `.claude/skills/pokemon-mcp/SKILL.md` · /pokemon-mcp driver
- `scripts/mcp/discover_api.sh` · admin REST API probe

docs · panel-shareable methodology

- `docs/rga-eval-methodology.md` · how we measure AI quality
- `docs/observability.md` · what we log + why
- `docs/caching-strategy.md` · cache trade-offs
- `docs/mcp-integration.md` · MCP setup + workflow
- `docs/detail-page.md` · Headless + React architecture

infrastructure-as-code (slide 6)

- `config/` · Coveo resource declarations (JSON / YAML)
- `scripts/bootstrap.sh` · one-command full provisioning
- `.github/workflows/rga-eval-daily.yml` · 06:00 UTC eval cron
- `.github/workflows/closed-loop-daily.yml` · 06:30 UTC auto-tune
- `observability/grafana-dashboard.json` · dashboard-as-code

github.com/benichou/coveo-pokemon-challenge